

The Vibe Coder's Playbook: A Framework for Crafting Prompt Scope Documents (PSDs)

Introduction: From Vague Vibes to Tangible Products

"Vibe coding" is an AI-driven software development practice that utilizes natural language prompts to generate functional code, dramatically accelerating the journey from concept to reality. However, transforming a high-level "vibe" into a successful product requires more than just a single command. The strategic foundation for this process is the **Prompt Scope Document (PSD)**. A PSD is a foundational framework that translates an abstract idea into a structured, actionable plan for an AI coding agent. By providing clear requirements, technical constraints, and essential context, the PSD ensures clarity, consistency, and a significantly higher probability of achieving a successful outcome. This guide provides a comprehensive framework for creating and utilizing PSDs to harness the full potential of agentic software development.

1. The Prompt Scope Document (PSD) Framework

1.1. Analyzing the Core Components of a PSD

A well-structured Prompt Scope Document (PSD) serves as the definitive blueprint for the AI agent, furnishing all necessary context in a predictable and digestible format. This structured approach is critical for minimizing AI guesswork and "hallucinations," which in turn leads to higher-quality code that accurately aligns with the project's vision.

1.1.1. Essential Sections

A comprehensive PSD is built upon several key informational components that collectively provide the AI with a complete picture of the task at hand. These sections are derived from best practices in product development and prompt engineering.

- **Project Vision & Objectives:** This section outlines the high-level goal, the target user, and the core problem the application is intended to solve. This gives the AI the "why" behind the "what," anchoring its technical decisions in user-centric goals.
- **Functional & UI/UX Requirements:** Here, the project is broken down into its smallest functional parts. This includes detailed feature descriptions, user flows, and specifications for the visual design, often linking to a mockup from a tool like Figma or v0.
- **Technical Specification:** This defines the chosen technology stack, key libraries, and any architectural constraints the AI must adhere to. Specifying popular, well-documented technologies yields better results as AI models are trained on public data.
- **Context & Knowledge Base:** This is where you front-load domain knowledge, effectively cloning your architectural expertise into the AI's context window before it writes a

single line of code. It includes relevant documentation, existing components, and established rules.

- **Success & Validation Criteria:** This section defines the testing requirements, performance benchmarks, and quality standards the final code must meet to be considered complete and successful.

1.1.2. The PSD as a "Context Bridge"

The PSD functions as a "Context Bridge," providing the AI agent with two critical types of information that are essential for it to operate effectively, especially when interacting with custom or specific backends.

- **Static Documentation:** These are the "rules of engagement" that explain *how* a system works. This includes API patterns, architectural constraints (e.g., "Use `subscriptions-transport-ws` for live messages"), and brand guidelines. This information is generally platform-specific and does not change frequently.
- **Dynamic Metadata:** This is the project-specific information that explains *what* is in the system at a given moment. This includes database schemas, component structures, and available API routes. This data is dynamic and changes as the application evolves.

This dual-channel structure forms the backbone of our reference template, ensuring both static rules and dynamic project data are systematically provided to the agent.

1.2. PSD Reference Template

The following template provides a ready-to-use structure for a Prompt Scope Document. It is designed to be populated with the specific details of your project, guiding the AI with precision and clarity.

Prompt Scope Document: [Project Name]

> **Instructions for Use:** Fill out each section with as much detail as possible. Remember: garbage in, garbage out. The clearer your plan, the smoother the execution.

1. Project Vision & Objectives

> Define the high-level goal. What problem are you solving? Who is the target user?

- **Project Goal:**
- **Target Audience:**
- **Core Value Proposition:**

2. Functional & UI/UX Requirements

> Break down the project into its smallest functional parts. For UI, provide a link to a mockup (Figma, v0, screenshot) and describe the visual hierarchy, color palette, and key components.

- **User Flow(s):** (e.g., 1. User lands on home screen. 2. User clicks 'Add Food'. 3. Camera opens...)
- **Key Features:**

- Feature A: [Detailed description of functionality]
- Feature B: [Detailed description of functionality]
- **UI/UX Mockup:** [Link to Dribbble, Figma, AI-generated image, etc.]
- **Design System & Components:**
 - **Color Palette:** (Primary, Secondary, Grays, Accents)
 - **Typography:** (Font Family, Weights, Sizes)
 - **Key Components:** (Buttons, Cards, Modals, Tables, etc. Describe their states: hover, focus, active).

3. Technical Specification

> Specify the technology stack. AI models are trained on public data; popular, well-documented technologies yield better results.

- **Frontend Framework:** (e.g., Next.js)
- **Styling:** (e.g., Tailwind CSS)
- **Backend/Database:** (e.g., Supabase for DB and Auth)
- **Deployment Platform:** (e.g., Vercel)
- **Architectural Constraints:** (e.g., "Use subscriptions-transport-ws for live messages," "Keep all secrets server-side only.")

4. Context & Knowledge Base

> Provide the AI with relevant documentation and rules. This is where you front-load knowledge to prevent common mistakes.

- **Relevant Files for Context:** (List specific files the AI should reference, e.g., ``components/ExistingButton.tsx``, ``tailwind.config.js``)
- **Cursor Rules / Guidelines File:** (Link to or paste rules, e.g., "Do not use magic strings, hex values, or px values. Replace all with Tailwind classes.")
- **Common Mistakes to Avoid:** (e.g., "Claude has a trait of modifying things you did not ask for. Do not change anything I did not ask for.")

5. Success & Validation Criteria

> Define how you will measure success. This includes testing requirements and performance benchmarks.

- **Testing Framework:** (e.g., pytest)
- **Required Code Coverage:** (e.g., "Ensure the ``widget_factory.py`` file has a code coverage of 95%.")
- **Validation Steps:** (e.g., "1. The navigation bar must be full-width. 2. The search modal must close when clicking outside of it. 3. The app must be responsive and theme-aware for light and dark modes.")

With this template as our blueprint, we can now explore the principles of effective prompting required to fill it out.

2. Best Practices for Prompting and Agent Interaction

2.1. Analyzing the Principles of Effective Prompting

The quality of a project is determined by the quality of its plan. In vibe coding, this means the PSD is only as effective as the prompts within it. The core principles of effective prompting—Clarity, Context, and Constraints—are the tools we use to populate the PSD template, transforming it from an empty document into a precise set of instructions for our AI agent.

2.1.1. The Three C's: Clarity, Context, and Constraints

A successful prompt delivers information to the AI with precision and purpose, guided by three fundamental concepts.

1. **Clarity:** Use precise, unambiguous language and avoid "fluffy" or vague descriptions. As advised in the Headway demo on Figma Make, structure instructions logically using bullet points or numbered lists, as if you were delegating tasks to a human intern. The AI needs to know exactly what to do.
2. **Context:** Provide brand-specific, user-specific, or project-specific information that anchors the AI's generation in the reality of your project. For example, instead of asking for a generic dashboard, specify that it is for "a modern fintech app called Mevolut."
3. **Constraints:** Define the boundaries for the AI's output. This includes specifying the target platform (e.g., "optimize for a mobile screen size"), required interaction patterns, or adherence to accessibility standards. Constraints reduce ambiguity and scope the AI's work effectively.

2.1.2. Formulating Prompts: Good vs. Bad Examples

The difference between a vague and a detailed prompt can mean the difference between a generic, unusable output and a near-perfect implementation.

Vague Prompt (To Avoid)	Effective Prompt (To Emulate)
"Make a signup screen."	"Create a sign-up form with fields for email and password, a 'Sign in with Google' button, and apply the Material 3 styling for all UI elements."

2.1.3. Establishing Rules and Guardrails

To provide persistent instructions that apply across multiple interactions, it is crucial to establish a "rules" file (e.g., `.cursor/rules` or a simple markdown document). This file acts as a set of guardrails for the AI, preventing common mistakes and enforcing best practices.

- Preventing unwanted style overrides: "Use built-in, standard Tailwind CSS design tokens instead of hardcoding values."
- Enforcing security best practices: "Keep secrets server-side only. Always validate & sanitize user input on the server."

- Mandating specific API patterns: "For async tasks, use a Mutation to start, a Subscription to listen, and parse the Result stream."
- Preventing the agent from "cheating" on tests: "Do NOT simplify tests of goals to succeed. Do not use mocking to pass a test instead of properly testing it."
- Controlling agent behavior: "Do not change anything I did not ask for. Only do what I told you."

These rules, combined with well-crafted prompts, form the basis of a disciplined operational framework for vibe coding.

3. The Vibe Coding Operational Framework

3.1. Evaluating the Vibe Coding Lifecycle

A well-defined PSD is the static plan; the Vibe Coding Operational Framework is the dynamic process of executing that plan. We operationalize vibe coding through a three-stage lifecycle: rigorous **Preparation** to de-risk execution, disciplined **Execution & Iteration** to ensure quality, and systematic **Troubleshooting** to maintain momentum.

3.2. Stage 1: Preparation (The Pre-Prompting Checklist)

Success in vibe coding is determined long before the first prompt is sent. The goal of the preparation stage is to prepare all necessary assets and establish a stable environment for the AI agent to work within.

- [] **Define the Vision:** Create the Prompt Scope Document (PSD) using the provided template. Leverage tools like Gemini to brainstorm product goals and structure your requirements.
- [] **Plan the UI/UX:** Create or source a design mockup. Decide on a design system and plan for reusable components from the very beginning to ensure consistency.
- [] **Prepare Design Files:** If pasting designs from a tool like Figma, ensure all layers are properly named and auto layout is correctly applied. Remove any hidden or unnecessary layers that could confuse the AI.
- [] **Bootstrap the Codebase:** Start with a solid, pre-configured starter kit that includes a design system, SEO essentials, and AI rules. AI is slow, expensive, and unreliable when it comes to generating a fresh codebase from scratch. A starter kit bypasses this weakness, providing a solid, maintainable foundation.
- [] **Set Up Version Control:** Initialize a Git repository. AI agents can and will screw up your codebase. Git is a non-negotiable safety net that prevents catastrophic errors from destroying weeks of work.

3.3. Stage 2: Execution & Iteration

The core of vibe coding is an interactive, multi-turn conversation with the AI agent. This process is about building incrementally, reviewing the output, and refining with targeted feedback.

1. **Initial Prompt:** Begin with a single, clear goal from your PSD, such as implementing one screen or one specific feature.

2. **Review the Output:** Carefully examine the code or prototype generated by the AI. Compare it directly against the requirements outlined in your PSD.
3. **Identify Flaws:** Systematically note any bugs, visual inconsistencies, or missing functionalities. For example: "The navigation is not full-width," "The delivery/pickup toggle is not working," or "The search modal is stuck open."
4. **Refine with Follow-up Prompts:** Write specific, targeted prompts to address each identified issue one by one. AI-generated code is a starting point, not a perfect final product. Expect to engage in a multi-turn dialogue to fix bugs, align visuals, and correct logic, just as you would guide a junior developer. If the AI begins to deviate, it is more effective to go back, refine the previous prompt, and resend it.
5. **Commit Changes:** Once a feature is implemented correctly and validated, commit the working code to your Git repository before proceeding to the next task.

3.4. Stage 3: Troubleshooting Common Issues

AI agents are powerful but imperfect tools that will inevitably encounter errors. Knowing how to handle these common problems is key to maintaining momentum.

- **Compilation or Build Errors:** The most straightforward issue to solve. Copy the full error message from your terminal or console and paste it directly into the prompt, instructing the AI to fix it.
- **Stubborn Bugs (The Rabbit Hole):** If the AI fails to resolve a bug after 2-3 attempts, it may be stuck. Instruct it to take a step back: ask it to provide an overview of the components involved, list the "top suspects" it believes are causing the error, and add logs. Provide the log output back to the AI for further analysis.
- **Context Window Limitations:** In a long chat session, an AI may begin to "forget" earlier instructions or established patterns. When you notice this, start a new chat window. Provide a brief summary of the current task and explicitly mention the specific files being worked on to re-establish context.
- **API Rate Limits:** High-intensity work can sometimes trigger API rate limits from the underlying AI model service. A practical workaround is to spin up a new account or generate a new API key. You will then need to re-acquaint the agent with the codebase, which can be done by asking it to read the contents of the project directory.

This operational framework provides a repeatable process for moving from a well-defined plan to a functional application.

4. Vibe Coding in Practice: Use Cases and Tooling

4.1. Assessing Project Goals: Prototypes vs. Production Code

The vibe coding methodology, guided by the initial PSD, must be adapted to the project's end goal. The approach for building a quick, disposable prototype is fundamentally different from that of crafting a scalable, maintainable production application.

Rapid Prototyping	Production-Ready Applications
Goal: Quickly validate ideas, demonstrate complex animations or interactions, and conduct early-stage user research.	Goal: Build scalable, maintainable, and secure code that can be deployed, monitored, and iterated upon by a development team.
Process: Focuses on achieving high visual and interactive fidelity. Often starts from a design mockup (e.g., in Figma) and uses specialized tools to bring it to life with minimal manual coding.	Process: Emphasizes a disciplined, iterative software development lifecycle because production code demands maintainability, security, and scalability—qualities that an unguided, single-shot AI process cannot reliably produce. It requires a solid starter kit, rigorous version control, and a focus on testing from day one.
Tools: Figma Make, Lovable.	Tools: Cursor, Claude Code, Windsurf.
Example: Building a clickable prototype of a new search feature to show stakeholders and gather feedback on the user experience.	Example: Building a full-stack developer worklog application with a database and authentication, then deploying it to a platform like Vercel.

4.2. Tooling Landscape for Vibe Coding

A diverse ecosystem of tools has emerged to support vibe coding, each optimized for different stages of the development process and for different types of users. The strategist's job is to match the right tool to the project's goal, the team's skillset, and the desired outcome, moving from low-fidelity planning tools to high-fidelity production IDEs as the project matures.

- **Figma Make:**
 - **Use Case:** High-fidelity, interactive prototyping directly from Figma designs.
 - **Strengths:** Tightly integrated with Figma, designer-friendly, excellent for demonstrating UI animations and interactions that are difficult to build with standard Figma prototyping. Can connect to a design system's styles.
 - **Target User:** Designers and product teams.
- **Cursor / Windsurf:**
 - **Use Case:** AI-native Integrated Development Environments (IDEs) for building and refactoring full applications.
 - **Strengths:** Allows for complex, multi-file code generation. Supports context through features like `.cursor/rules` and referencing the entire codebase. More robust and scalable than web-based tools.

- **Target User:** Developers and technical "vibe coders."
- **Claude Code / Claude Engineer:**
 - **Use Case:** Terminal-based, agentic development for more autonomous and complex coding tasks.
 - **Strengths:** Extremely powerful and can perform multi-file operations. Can be instructed to follow a multi-step plan. Knows the entire application structure.
 - **Target User:** Advanced users and developers comfortable with the command line.
- **Gemini (in Google AI Studio):**
 - **Use Case:** Planning, brainstorming, and prompt refinement.
 - **Strengths:** Large context window is ideal for structuring thoughts, outlining product goals, creating a detailed PSD, and reviewing generated code for security flaws or bad patterns. Can also be used as a real-time coding assistant via screen sharing.
 - **Target User:** All users, especially in the planning phase.
- **Lovable:**
 - **Use Case:** Zero-to-one greenfield development and prototyping.
 - **Strengths:** Excellent for non-technical users to explore and prototype ideas, effectively communicating requirements to an engineering team. Can be taught rules via a "Knowledge Base."
 - **Target User:** Non-technical founders, product managers, and sales teams.

The PSD and the Operational Framework are two halves of a whole: the static plan and the dynamic process. This dual approach is what elevates "vibe coding" from a novel trick to a legitimate production methodology. When guided by a structured plan and a disciplined workflow, this paradigm can dramatically accelerate the creation of both innovative prototypes and production-grade applications.